MODELARE ȘI SIMULARE

Săptămâna 3. Procese stocastice: plimbări aleatoare și lanțuri Markov

Suport de curs · Anul III, Calculatoare și Tehnologia Informației

Ș.l. dr. ing. Cătălin Iapă

Obiective

1. să simulați o plimbare aleatoare și să explicați de ce distanța tipică crește ca √n;
2. să legați regulile microscopice (pașii aleatori) de comportamentul macroscopic (difuzia);
3. să descrieți un sistem ca lanț Markov: stări, graf, matrice de tranziție, distribuție inițială;
4. să calculați distribuția după n pași și distribuția staționară;
5. să recunoașteți stările absorbante, periodice și lanțurile ergodice;
6. să explicați PageRank, generarea de text cu n-grame și ideea algoritmului Metropolis.

Cuvinte cheie: *proces stocastic, plimbare aleatoare, difuzie, deplasare pătratică medie, ruina jucătorului, proprietatea Markov, matrice de tranziție, distribuție staționară, lanț ergodic, stare absorbantă, matrice fundamentală, PageRank, n-grame, disponibilitate, MCMC, algoritmul Metropolis*

1. Plimbarea aleatoare și difuzia

1.1. De la variabile aleatoare la procese stocastice

În săptămâna 2 am lucrat cu variabile aleatoare **independente**: fiecare punct al estimării lui π era nou. În sistemele reale însă, starea de azi pornește de la starea de ieri: numărul de biciclete dintr-o stație la minutul următor depinde de numărul din minutul curent. Un **proces stocastic** este o familie de variabile aleatoare {X(t)} indexate de timp, care descrie evoluția unui sistem supus întâmplării; mulțimea valorilor posibile se numește **spațiul stărilor** (Holban, cap. 5). O rulare a simulării produce **o traiectorie** a procesului, iar multe rulări ne dau distribuția traiectoriilor.

Timpul poate fi discret (pași 0, 1, 2, …) sau continuu, iar stările pot fi discrete sau continue. Obținem patru familii: lanțuri Markov și plimbări aleatoare (timp și stări discrete – tema acestei săptămâni), serii de timp (timp discret, stări continue), cozi și procesul Poisson (timp continuu, stări discrete – săptămâna 4) și mișcarea browniană (timp și stări continue).

1.2. Plimbarea aleatoare simplă

Pornim din $X_0 = 0$; la fiecare pas aruncăm o monedă: $X_{n+1} = X_n + 1$ cu probabilitatea p și $X_{n+1} = X_n - 1$ cu probabilitatea 1 − p. Pentru p = ½ plimbarea este **simetrică**. Poziția după n pași este suma a n pași independenți $\xi_i = \pm 1$. Un pas are media 0 și dispersia $E[\xi^2] - 0^2 = 1$; dispersiile variabilelor independente se adună, deci:

$$E[X_n] = 0, \quad Var(X_n) = n, \quad \sigma_n = \sqrt{n}$$

Distanța tipică față de start crește doar ca **√n**: după 100 de pași ≈ 10, după 10 000 de pași ≈ 100. Este aceeași lege ca eroarea Monte Carlo din săptămâna 2. Figura 1 arată cinci traiectorii: aceeași regulă produce drumuri foarte diferite, iar unele rămân mult timp de aceeași parte a lui 0 – plimbarea nu are memorie și nu „știe” unde este zero. Într-o simulare cu 10 000 de plimbări a câte 100 de pași am obținut media 0,10, abaterea standard 9,99 și distanța medie $|X_{100}| = 8{,}02$ (teoretic $\sqrt{(2n/\pi)} \approx 7{,}98$).

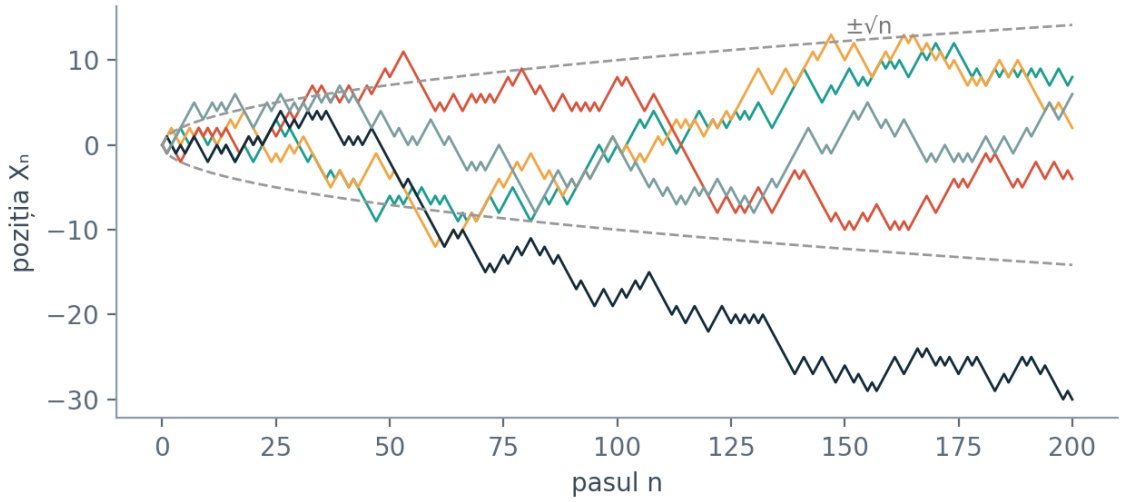


Figura 1. Cinci plimbări aleatoare simetrice de câte 200 de pași; liniile punctate arată ±√n

```
import numpy as np
rng = np.random.default_rng(2)
steps = rng.choice([-1, 1], size=(10_000, 100))    # o linie = o plimbare
x = steps.sum(axis=1)                               # poziția finală
print(x.mean(), x.std())                            # ≈ 0 și ≈ 10
```

Poziția finală este o sumă de variabile independente, deci, conform teoremei limită centrale, are distribuția aproximativ normală N(0, n) (Figura 2). Doar 16 din 10 000 de plimbări au ajuns mai departe de ±30, adică de trei abateri standard.

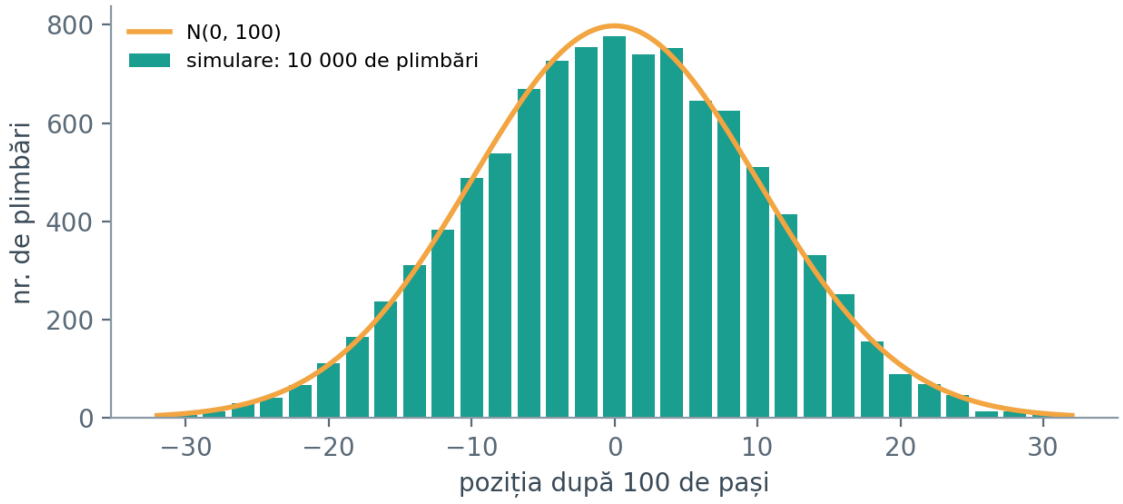


Figura 2. Poziția după 100 de pași pentru 10 000 de plimbări, comparată cu aproximarea normală N(0, 100)

Dacă $p \neq ½$, fiecare pas are media $2p − 1$, iar $E[X_n] = n(2p − 1)$, $\mathrm{Var}(X_n) = 4np(1 − p)$. Pentru $p = 0{,}55$, după 200 de pași ne așteptăm la +20, cu o abatere standard de aproximativ 14. Tendința crește ca n, zgomotul doar ca √n, deci pe termen lung tendința domină; în schimb, pe termen scurt, o tendință mică este ascunsă de zgomot și cere multe date ca să fie observată.

1.3. Ruina jucătorului

Un jucător pornește cu k lei, pariază câte 1 leu și se oprește la 0 (ruină) sau la N (țintă). Stările 0 și N sunt **absorbante**: odată atinse, procesul se oprește. Într-un joc corect, probabilitatea de a atinge ținta este k/N,

iar numărul mediu de mize este k(N − k). Dacă p < ½, cu r = (1 − p)/p, probabilitatea devine $(1 - r^k)/(1 - r^N)$. Simularea a 10 000 de jocuri cu k = 10 și N = 20 a dat 0,495 și 98,9 mize (teoretic 0,5 și 100). La ruletă (p = 18/37), probabilitatea scade la 0,368, iar pentru k = 50, N = 100 la doar 0,063 – deși un singur pariu de 50 de lei ar avea 48,6% șanse. Un dezavantaj mic devine uriaș când jocul durează mult.

1.4. Plimbarea în 2D și difuzia

În exemplul profesorului Cernazean (Lecture 3), plimbătorul stă pe o rețea pătrată și la fiecare pas alege una din 5 mutări: dreapta, stânga, sus, jos sau pe loc, fiecare cu probabilitatea 1/5. Un singur traseu este neregulat și imprevizibil, dar un nor de 1 000 de plimbători pornit din origine rămâne centrat și se lărgește cu o rază care crește ca √t (Figura 3, stânga). Mărimea potrivită este **deplasarea pătratică medie** $\langle r^2(t) \rangle$, media pătratului distanței față de start:

$$\langle r^2(t) \rangle = 4\,D\,t \quad (\text{în } 2D); \quad \text{în general } \langle r^2 \rangle = 2\,d\,D\,t$$

D este **coeficientul de difuzie**. Pentru modelul cu 5 mutări, $E[\Delta x^2] = E[\Delta y^2] = 2/5$, deci D = 1/5 și $\langle r^2(t) \rangle = 0{,}8 \cdot t$. Simularea a 10 000 de plimbători a dat $\langle r^2 \rangle = 8{,}11$ la t = 10 și 80,13 la t = 100 (Figura 3, dreapta). Cu N = 100, 1 000, 10 000 și 100 000 de plimbători, estimarea la t = 100 a fost 92,8; 81,0; 78,9; 80,5 – eroarea scade ca 1/√N, ca în orice estimare Monte Carlo.

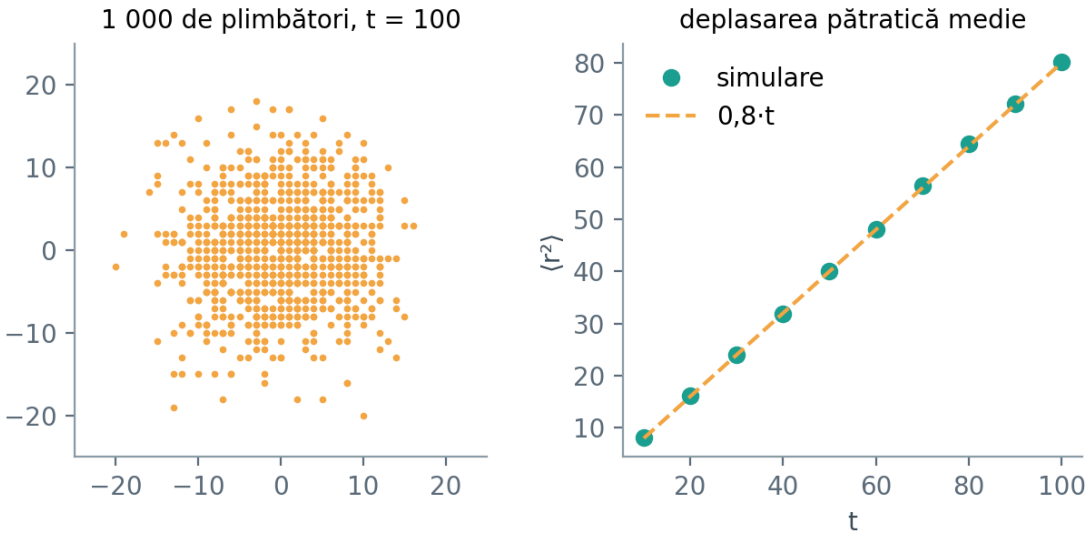


Figura 3. Stânga: pozițiile a 1 000 de plimbători după 100 de pași. Dreapta: $\langle r^2 \rangle$ măsurat pe 10 000 de plimbători și legea 0,8·t

Aceasta este legătura dintre **micro** și **macro**: fiecare particulă face o plimbare aleatoare fără niciun „plan”, dar concentrația se netezește după o lege precisă, ecuația difuziei $\partial c/\partial t = D\nabla^2 c$. Legea există doar la nivelul populației – un exemplu de **emergență**, pe care îl vom revedea la automatele celulare (S8) și la agenți (S9). Din $t \sim r^2$ rezultă că difuzia este lentă pe distanțe mari: de 10 ori mai departe înseamnă de 100 de ori mai mult timp.

Tabelul 1. Plimbarea aleatoare: legile pe scurt

Model	Media poziției	Dispersia	Distanța tipică
1D simetrică (±1, p = ½)	0	n	√n
1D cu tendință (p ≠ ½)	n(2p − 1)	4np(1 − p)	√(4np(1 − p))

Model	Media poziției	Dispersia	Distanța tipică
2D, 5 mutări (Cernazean)	(0, 0)	$\langle r^2 \rangle = 0{,}8 \cdot t$	$\sqrt{(0{,}8 \cdot t)}$
d dimensiuni	0	$\langle r^2 \rangle = 2dDt$	$\sqrt{(2dDt)}$

1.5. Mișcarea browniană și aplicații

Limita plimbării aleatoare cu pași foarte mici și foarte deși este **mișcarea browniană**: B(t) este continuă în timp și B(t) ~ N(0, t). Robert Brown a observat în 1827 mișcarea haotică a unor particule din polen în apă; Einstein a explicat-o în 1905 prin ciocnirile cu moleculele, iar Perrin a confirmat-o experimental. Tot în 1905, Karl Pearson a folosit pentru prima dată termenul „random walk", iar Bachelier modelase încă din 1900 prețurile de la bursă ca plimbare aleatoare. Modelul modern de preț înmulțește prețul cu exponențiala unui randament normal (plimbare aleatoare geometrică); atenție însă, randamentele reale au cozi grele.

Pólya a arătat în 1921 că plimbarea simetrică revine sigur în punctul de plecare în 1D și 2D, dar în 3D doar cu probabilitatea ≈ 0,34. În informatică, plimbările aleatoare pe grafuri se folosesc pentru eșantionarea rețelelor mari, în sisteme de recomandare (de exemplu Pinterest Pixie, 2018), pentru reprezentări ale nodurilor (DeepWalk, 2014; node2vec, 2016) și în algoritmi randomizați (algoritmul lui Papadimitriou pentru 2-SAT, 1991). Modelele de **difuzie** din AI generativ (Sohl-Dickstein et al., 2015; Ho et al., 2020) adaugă pas cu pas zgomot gaussian unei imagini – un lanț Markov, practic o plimbare aleatoare în spațiul imaginilor – și antrenează o rețea neurală să parcurgă drumul invers, de la zgomot la o imagine nouă.

De reținut

Plimbarea aleatoare: $E[X_n] = 0$ și $\sigma = \sqrt{n}$ (simetrică); $E[X_n] = n(2p - 1)$ cu tendință. În 2D, $\langle r^2 \rangle = 4Dt$: difuzia apare din reguli locale. Ruina jucătorului: $P = k/N$ în jocul corect. Poziția de mâine depinde doar de poziția de azi – plimbarea este un lanț Markov.

2. Lanțuri Markov

2.1. Proprietatea Markov

Un **lanț Markov** este un proces cu timp discret și stări discrete care are **proprietatea Markov**: $P(X_{n+1} = j \mid X_0, \ldots, X_{n-1}, X_n = i) = P(X_{n+1} = j \mid X_n = i) = P_{ij}$. Pentru a prezice „mâine" trebuie să știm doar „azi"; „ieri" nu aduce informație în plus (Holban, cap. 5.1). Dacă P_{ij} nu depinde de n, lanțul este **omogen** – toate lanțurile din acest curs sunt omogene. Multe sisteme au proprietatea Markov dacă alegem bine starea (poziția unei plimbări, numărul de biciclete dintr-o stație, starea unui server). Când sistemul are memorie, **extindem starea**: dacă mâine depinde de azi și de ieri, folosim ca stare perechea (ieri, azi). Starea trebuie să conțină tot ce e necesar pentru a prezice pasul următor, iar costul este creșterea numărului de stări.

2.2. Graf și matrice de tranziție

Exemplul pietonului aleator (Holban, după Häggström): un oraș are 4 colțuri de stradă v1–v4 așezate în pătrat; pietonul pornește din v1 și la fiecare pas aruncă o monedă – la cap merge în sens orar, la pajură în sens trigonometric. Graful are săgeți cu probabilitatea ½ între colțurile vecine, iar matricea de tranziție are pe rândul i probabilitățile de a pleca din starea i:

$$P = [\,0\ \tfrac{1}{2}\ 0\ \tfrac{1}{2}\,;\ \tfrac{1}{2}\ 0\ \tfrac{1}{2}\ 0\,;\ 0\ \tfrac{1}{2}\ 0\ \tfrac{1}{2}\,;\ \tfrac{1}{2}\ 0\ \tfrac{1}{2}\ 0\,]$$

O **matrice de tranziție** (stocastică) are $P_{ij} \geq 0$ și fiecare **rând** cu suma 1. Împreună cu distribuția inițială $\pi^{(0)}$, ea descrie complet lanțul. Graful și matricea sunt două reprezentări ale aceluiași model: săgețile lipsă sunt zerouri în matrice.

2.3. Distribuția după n pași

Dacă $\pi^{(n)}$ este vectorul-linie al probabilităților stărilor la pasul n, atunci $\pi_j^{(n+1)} = \Sigma_i\, \pi_i^{(n)} \cdot P_{ij}$, adică $\pi^{(n+1)} = \pi^{(n)}P$ și, aplicând de n ori (Holban, teorema 1):

$$\pi^{(n)} = \pi^{(0)} \cdot P^n$$

Elementul $(P^n)_{ij}$ este probabilitatea de a ajunge în j după n pași, pornind din i, iar $P^{m+n} = P^m P^n$ (Chapman–Kolmogorov). Exemplul vremii (Holban, 3a): stările soare și ploaie, P = [0,9 0,1; 0,5 0,5]. Dacă azi e soare, $\pi^{(0)}$ = (1, 0), $\pi^{(1)}$ = (0,9; 0,1), $\pi^{(2)}$ = (0,86; 0,14). Tabelul 2 și Figura 4 arată că, pornind de la soare sau de la ploaie, probabilitatea de soare converge rapid la aceeași valoare, 5/6 ≈ 0,8333: lanțul își uită punctul de plecare.

Tabelul 2. Probabilitatea de soare peste n zile

Ziua n	dacă azi e soare	dacă azi plouă
0	1,0000	0,0000
1	0,9000	0,5000
2	0,8600	0,7000
3	0,8440	0,7800
4	0,8376	0,8120
5	0,8350	0,8248
10	0,8334	0,8332
20	0,8333	0,8333

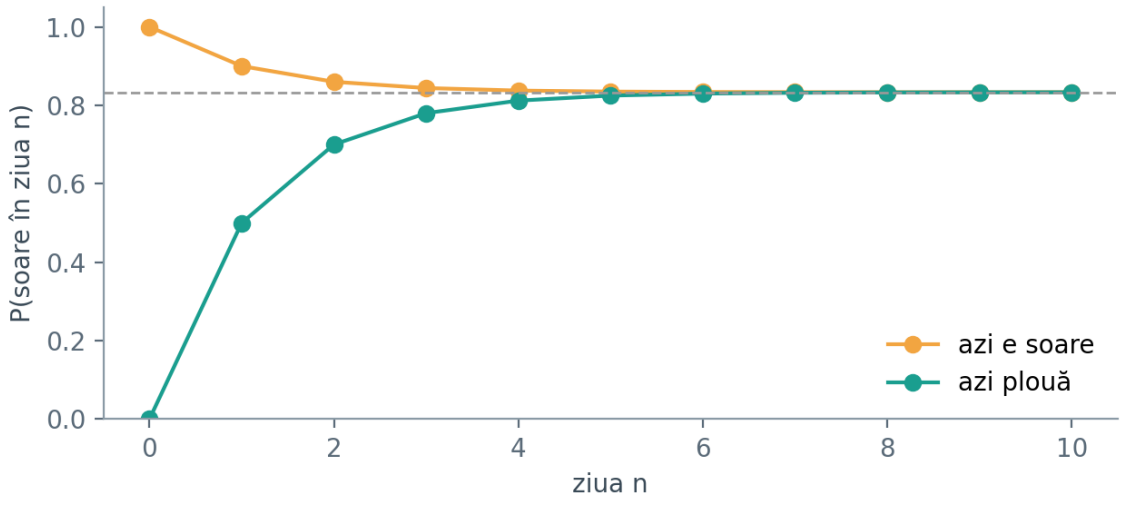


Figura 4. Modelul vremii: P(soare în ziua n) pornind de la o zi însorită și de la o zi ploioasă; linia punctată este 5/6

Proprietatea Markov permite calculul probabilității unei secvențe ca produs: $P(x_0, \ldots, x_n) = \pi^{(0)}(x_0) \cdot P_{x_0 x_1} \cdots P_{x_{n-1} x_n}$. Pentru modelul cu trei stări (ploaie, înnorat, soare) cu matricea [0,4 0,3 0,3; 0,2 0,6 0,2; 0,1 0,1 0,8] (Rabiner, 1989; Holban, 3b), secvența S S S P P S Î S, pornind de la soare, are probabilitatea $1 \cdot 0{,}8 \cdot 0{,}8 \cdot 0{,}1 \cdot 0{,}4 \cdot 0{,}3 \cdot 0{,}1 \cdot 0{,}2 = 1{,}536 \cdot 10^{-4}$. (În materialul vechi etichetele stărilor sunt inversate față de matrice; aici s-a folosit ordinea din sursa originală.)

Simularea unui lanț Markov este simplă: la fiecare pas generăm starea următoare dintr-o distribuție discretă dată de rândul stării curente – exact transformarea inversă din S2. Pe un drum de 10 000 de zile, fracțiunea zilelor cu soare a fost 0,8378. Convergența este mai lentă decât la eșantioane independente, pentru că zilele consecutive sunt corelate.

```
rng = np.random.default_rng(10)
P = np.array([[0.9, 0.1], [0.5, 0.5]])
x, sunny = 0, []
for _ in range(10_000):
    x = rng.choice(2, p=P[x])      # starea următoare
    sunny.append(x == 0)
print(np.mean(sunny))                # ≈ 0.833
```

2.4. Distribuția staționară

O **distribuție staționară** este un vector de probabilități π cu $\pi = \pi P$ și $\Sigma \pi_i = 1$ (Holban, cap. 5.5): dacă pornim din π, rămânem în π. Interpretarea practică: π_i este fracțiunea de timp petrecută în starea i pe termen lung, după regimul tranzitoriu. Pentru două stări, cu $a = P(1 \to 2)$ și $b = P(2 \to 1)$, echilibrul fluxurilor $\pi_1 a = \pi_2 b$ dă:

$$\pi_1 = b / (a + b), \quad \pi_2 = a / (a + b)$$

Pentru vreme: $\pi(\text{soare}) = 0{,}5/0{,}6 = 5/6$. Pentru mai multe stări rezolvăm sistemul $\pi(P - I) = 0$ împreună cu $\Sigma \pi_i = 1$ (o ecuație din sistem este mereu redundantă). În exemplul A al profesorului Holban, Q = [0 1 0; ¼ ¼ ½; 0 ½ ½], ecuațiile dau $\pi_2 = 4\pi_1$ și $\pi_3 = \pi_2$, deci $\pi = (1/9, 4/9, 4/9)$. Formula compactă din Holban, $\pi = 1 \cdot (Q + E - I)^{-1}$, unde E are toate elementele 1, dă același rezultat. Din π rezultă indici de performanță (Tabelul 3).

Tabelul 3. Indici de performanță din distribuția staționară (exemplul A, starea 3, un pas = 50 u.t.)

Indice	Formula	Valoare
Fracțiunea de timp / gradul de utilizare	π_i	4/9 ≈ 44%
Timpul mediu de ședere în i	$1/(1 - P_{ii})$ pași	2 pași = 100 u.t.
Timpul mediu de revenire în i	$1/\pi_i$ pași	9/4 pași = 112,5 u.t.

Observație: timpul de ședere depinde de probabilitatea de a **rămâne** în stare, P_{ii}, nu de π_i; în materialul vechi apare calculul cu π_i (90 u.t.), de aceea valoarea de aici (100 u.t.) diferă.

2.5. Clasificarea stărilor și teorema ergodică

Starea j este **accesibilă** din i dacă există un drum de probabilitate pozitivă de la i la j. Lanțul este **ireductibil** dacă orice stare e accesibilă din oricare alta. O stare este **recurentă** dacă revenim sigur în ea și **tranzitorie** dacă la un moment dat o părăsim definitiv; este **absorbantă** dacă $P_{ii} = 1$. O stare este **periodică** cu perioada $d > 1$ dacă revenirile sunt posibile doar la multipli de d pași. Pietonul lui Holban are perioada 2: la pașii pari poate fi doar în v1 sau v3, iar $\pi^{(n)}$ oscilează (Figura 5). Dacă îl lăsăm să rămână pe loc cu probabilitatea ½

(„pietonul leneș”), lanțul devine aperiodic și converge la (¼, ¼, ¼, ¼). (Exemplul de lanț „periodic” cu trei stări din materialul vechi are cicluri de lungime 2 și 3, deci este de fapt aperiodic.)

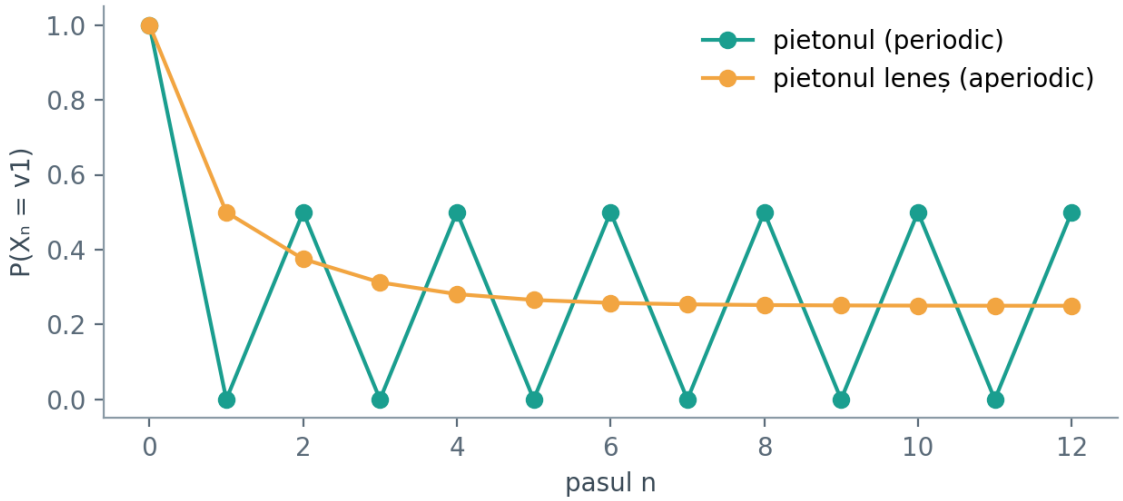


Figura 5. $P(X_n = v1)$ pentru pietonul periodic și pentru pietonul leneș

Teorema ergodică (fără demonstrație): dacă lanțul este finit, ireductibil și aperiodic (**ergodic**), atunci (1) există o unică distribuție staționară π, cu toate $\pi_i > 0$; (2) $\pi^{(n)} \to \pi$ indiferent de punctul de plecare; (3) fracțiunea de timp petrecută în i pe un singur drum lung converge la π_i. Punctul (3) justifică simularea cu un singur drum lung.

2.6. Lanțuri absorbante

Un lanț **absorbant** are cel puțin o stare absorbantă, accesibilă din orice stare; celelalte stări sunt tranzitorii. Aici interesează probabilitățile de absorbție și timpul până la absorbție. Scriem matricea cu blocurile Q (tranzitorii → tranzitorii) și R (tranzitorii → absorbante); **matricea fundamentală** $N = (I - Q)^{-1} = I + Q + Q^2 + \ldots$ conține numărul mediu de vizite, B = NR probabilitățile de absorbție, iar t = N·1 numărul mediu de pași până la absorbție.

Exemplu: vizita pe un magazin online, cu stările tranzitorii Acasă, Produs, Coș, Plată și stările absorbante Cumpărat și Plecat (Acasă → Produs 0,6, → Plecat 0,4; Produs → Acasă 0,2, → Produs 0,2, → Coș 0,3, → Plecat 0,3; Coș → Produs 0,2, → Plată 0,5, → Plecat 0,3; Plată → Cumpărat 0,7, → Coș 0,1, → Plecat 0,2). Calculul dă: 10,8% din vizitele care încep pe Acasă se termină cu o cumpărare, cu 2,63 pagini vizitate în medie; dintre vizitatorii ajunși la Plată cumpără 74,1%. Simularea a 20 000 de vizite a confirmat valorile (10,9% și 2,62).

2.7. Estimarea matricei din date

În practică, matricea se estimează din date: numărăm tranzițiile n_{ij} și împărțim la totalul pe rând, $\hat{P}_{ij} = n_{ij}/n_i$. Pe jurnalul simulat al unui server (1 000 de zile, stările OK, Degradat, Căzut), rândul OK a avut 902 de observații și estimări apropiate de valorile adevărate, dar rândul Căzut doar 31, cu $\hat{P}$(Căzut → OK) = 0,484 față de 0,5: puține date înseamnă eroare mare. Holban arată și estimarea din distribuțiile observate: un utilaj are p(0) = (1, 0), p(1) = (0,9; 0,1), p(2) = (0,82; 0,18), de unde Q = [0,9 0,1; 0,1 0,9]; proporția pieselor care necesită intervenții depășește 40% după 8 ani de exploatare (41,6%) și tinde la 50%.

De reținut

Lanț Markov = stări + matrice de tranziție (rânduri cu suma 1) + distribuție inițială. $\pi^{(n)} = \pi^{(0)}P^n$. Distribuția staționară $\pi = \pi P$ dă fracțiunea de timp; pentru 2 stări $\pi_1 = b/(a + b)$. Ergodic ⇒ π unică și atinsă din orice start; periodic ⇒ $\pi^{(n)}$ oscilează. Lanțuri absorbante: $N = (I - Q)^{-1}$, $B = NR$. Matricea se estimează prin numărarea tranzițiilor.

3. Lanțuri Markov în informatică

3.1. PageRank

Brin și Page (1998) au ordonat paginile web imaginând un **internaut aleator**: dă click la întâmplare pe unul din linkurile paginii curente, iar din când în când sare la o pagină oarecare. PageRank-ul unei pagini este fracțiunea de timp petrecută pe ea pe termen lung – distribuția staționară a lanțului. Lanțul definit doar de linkuri are capcane (cicluri din care nu se iese) și fundături (pagini fără linkuri), de aceea se adaugă **teleportarea**: cu probabilitatea d = 0,85 se urmează un link, cu 0,15 se sare la o pagină aleasă uniform. Matricea Google este:

$$G = d \cdot M + (1 - d)/n \cdot E, \qquad \pi = \pi \cdot G$$

unde $M_{ij} = 1/$(numărul de linkuri din i) dacă i → j. Teleportarea face lanțul ireductibil și aperiodic, deci π este unică. Pentru un mini-web cu legăturile A → B, C; B → C; C → A; D → A, C; E → A, D, iterația $\pi \leftarrow \pi G$ (power iteration) converge sub 10^{-6} în 26 de iterații și dă A = 0,372, B = 0,188, C = 0,366, D = 0,043, E = 0,030 (Figura 6). A și C primesc câte 3 linkuri, dar A este puțin mai sus, pentru că C își trimite tot scorul spre A; E nu primește niciun link și are doar partea din teleportare, 0,15/5 = 0,03. Fără teleportare, internautul rămâne prins în ciclul A–B–C, iar D și E primesc 0. Pentru web-ul real, M este rară, iar câteva zeci de iterații sunt suficiente, deoarece eroarea scade cel puțin cu factorul d la fiecare iterație.

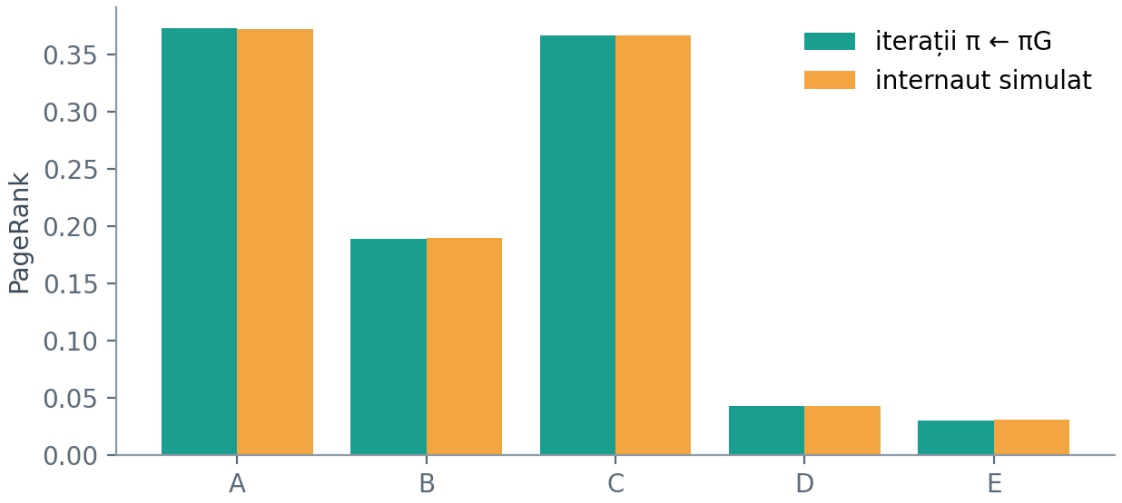


Figura 6. PageRank-ul celor 5 pagini: iterații $\pi \leftarrow \pi G$ și frecvențele unui internaut simulat (100 000 de click-uri)

3.2. Lanțuri Markov și limbajul

Prima aplicație a lanțurilor a fost făcută de Markov însuși (1913): a numărat primele 20 000 de litere din „Evgheni Oneghin” de Pușkin și a găsit P(vocală | vocală) = 0,128 și P(vocală | consoană) = 0,663, deci literele

nu sunt independente. Distribuția staționară a lanțului cu două stări dă fracțiunea de vocale 0,663/(0,663 + 0,872) = 0,432. Shannon (1948) a generat texte „aproape englezești” cu n-grame de litere și de cuvinte.

Un **model cu n-grame** presupune că probabilitatea cuvântului următor depinde doar de ultimele n − 1 cuvinte; probabilitățile se estimează prin numărare: $\hat{P}(b \mid a) = \text{nr}(a \text{ urmat de } b)/\text{nr}(a)$. Pentru n = 2 (bigrame), starea este ultimul cuvânt și avem un lanț Markov pe cuvinte. Pe un corpus de 82 de cuvinte despre acest curs, un generator cu bigrame a produs propoziții ca „simularea rulează modelul de starea următoare depinde doar de multe ori”: fiecare pereche de cuvinte este plauzibilă, dar propoziția nu mai are sens, pentru că modelul vede un singur cuvânt înapoi. Cu n mai mare textul devine mai coerent, dar începe să copieze corpusul, iar numărul de combinații crește exploziv (50 000 de cuvinte dau $2{,}5 \cdot 10^9$ bigrame posibile).

Un **model mare de limbaj** (LLM) calculează probabilitatea următorului token cu o rețea neurală, dintr-un context de mii – milioane de tokeni, și generalizează prin reprezentări învățate. Generarea este tot „un token pe rând”, dar starea este întregul context, deci modelul nu poate fi scris ca un tabel de tranziții. Istoric: n-grame în recunoașterea vorbirii și traducere (1980–2010), primul model neural de limbaj (Bengio et al., 2003), arhitectura Transformer (2017), LLM-urile actuale.

3.3. Fiabilitate și disponibilitate

Un server este OK sau căzut; într-o zi, un server funcțional cade cu probabilitatea a = 0,01, iar unul căzut este reparat cu b = 0,5. Timpul mediu între defecte este MTBF = 1/a = 100 de zile, iar timpul mediu de reparație MTTR = 1/b = 2 zile. **Disponibilitatea** este fracțiunea de timp în care serverul funcționează, adică π(OK):

$$A = b / (a + b) = MTBF / (MTBF + MTTR)$$

Aici A = 100/102 ≈ 98,04% (simulare pe 100 de ani: 97,78%), adică aproximativ 172 de ore de oprire pe an. Cu două servere independente, sistemul e oprit doar când ambele sunt căzute: $1 - (1 - A)^2 \approx 99{,}96\%$, adică ≈ 3,4 ore pe an. Presupunerea de independență este critică: defectele cu cauză comună (aceeași sursă de curent, aceeași actualizare greșită) anulează câștigul.

3.4. MCMC și algoritmul Metropolis

Până acum aveam un lanț și căutam π. Metodele **MCMC** (Markov chain Monte Carlo) întorc problema: dată o distribuție f cunoscută doar până la o constantă (situație tipică în statistica bayesiană), construim un lanț Markov a cărui distribuție staționară este f și îl rulăm mult timp. **Algoritmul Metropolis** (Metropolis, Rosenbluth, Rosenbluth, Teller și Teller, 1953): din starea x propunem y = x + zgomot; acceptăm y cu probabilitatea min(1, f(y)/f(x)), altfel rămânem în x; aruncăm primele stări (perioada de încălzire). Constanta de normare se simplifică în raport, deci nu trebuie cunoscută.

```
rng = np.random.default_rng(16)
x, xs = 0.0, []
for i in range(50_000):
    y = x + rng.normal(0, 1.0)          # propunere
    if rng.random() < f(y) / f(x):      # acceptare cu min(1, f(y)/f(x))
        x = y
    xs.append(x)
xs = np.array(xs[1000:])                # fără perioada de încălzire
```

Pentru un amestec de două normale (30% în jurul lui −2, 70% în jurul lui +2), rata de acceptare a fost 63,7%, media eșantioanelor 0,777 (teoretic 0,8) și fracțiunea valorilor pozitive 0,693 (teoretic 0,7) – Figura 7. Mărimea pasului contează: cu σ = 0,1 s-au acceptat 96% din propuneri, dar lanțul abia se mișcă; cu σ = 10, doar 17%. Eșantioanele MCMC sunt corelate, deci valorează mai puțin decât tot atâtea eșantioane

independente. MCMC se folosește în statistica bayesiană (Stan, PyMC), fizică (modelul Ising), criptanaliză (Diaconis, 2009, descrie decodarea unui mesaj cifrat prin substituție cu MCMC și statistici de bigrame) și învățare automată.

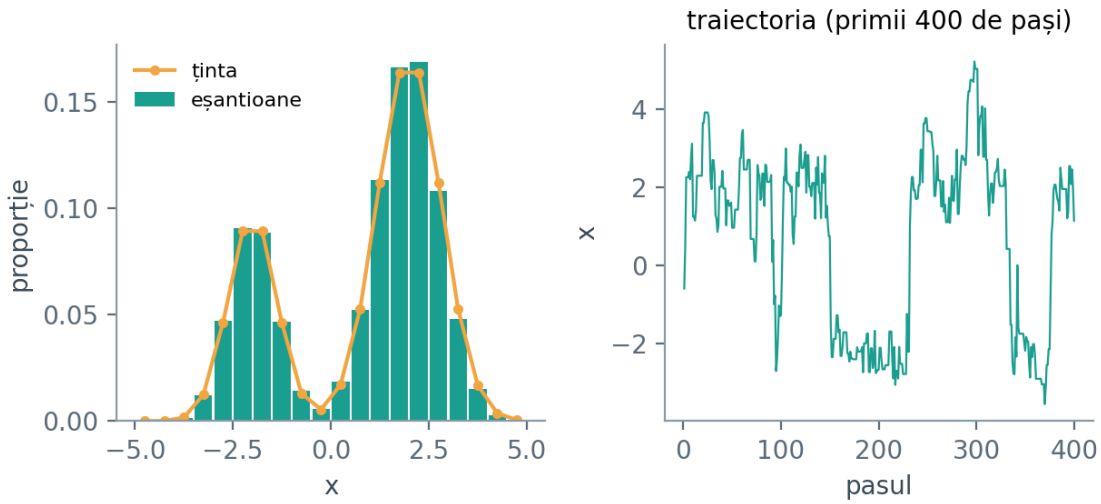


Figura 7. Algoritmul Metropolis: histograma eșantioanelor comparată cu distribuția țintă (stânga) și traiectoria primilor 400 de pași (dreapta)

3.5. Teaser: procese de decizie Markov

Un **proces de decizie Markov** (MDP) adaugă acțiuni și recompense: în starea s, agentul alege acțiunea a, sistemul trece în s' cu probabilitatea P(s' | s, a) și agentul primește recompensa r(s, a). Scopul este o politică a = π(s) care maximizează recompensa totală (Bellman, 1957). Exemplu: autoscaling în cloud – starea este încărcarea, acțiunea este adăugarea sau eliminarea unui server, iar recompensa penalizează costul și latența. MDP-urile sunt baza învățării prin întărire, iar agenții se antrenează de obicei în simulatoare (săptămâna 10).

De reținut

PageRank = distribuția staționară a internautului aleator, cu teleportare (d = 0,85). n-grame = lanț Markov pe cuvinte, estimat prin numărare; LLM-urile folosesc tot contextul. Disponibilitate = MTBF/(MTBF + MTTR). Metropolis: acceptă cu min(1, f(y)/f(x)); aruncă perioada de încălzire; eșantioanele sunt corelate.

Glosar RO – EN

Română	English
proces stocastic	stochastic process
traiectorie	sample path, trajectory
plimbare aleatoare	random walk
tendință	drift
ruina jucătorului	gambler's ruin

Română	English
deplasare pătratică medie	mean squared displacement
coeficient de difuzie	diffusion coefficient
mișcare browniană	Brownian motion
emergență	emergence
proprietatea Markov	Markov property
lanț Markov omogen	time-homogeneous Markov chain
matrice de tranziție	transition matrix
distribuție inițială	initial distribution
distribuție staționară	stationary distribution
ireductibil	irreducible
periodic / aperiodic	periodic / aperiodic
stare recurentă / tranzitorie	recurrent / transient state
stare absorbantă	absorbing state
lanț ergodic	ergodic chain
matrice fundamentală	fundamental matrix
factor de amortizare	damping factor
model cu n-grame	n-gram model
disponibilitate	availability
timp mediu între defecte	mean time between failures (MTBF)
timp mediu de reparație	mean time to repair (MTTR)
perioadă de încălzire	burn-in period
rată de acceptare	acceptance rate
proces de decizie Markov	Markov decision process (MDP)

Întrebări de autoevaluare

1. O plimbare aleatoare simetrică face 400 de pași de ±1. Distanța tipică (abaterea standard) față de start este:

a) 0

b) 20

c) 200

d) 400

2. În simularea 2D, $\langle r^2 \rangle \approx 80$ la t = 100. Cât estimăm pentru $\langle r^2 \rangle$ la t = 400?

a) 160

b) 80

c) 320

d) 6 400

3. Proprietatea Markov spune că:

a) starea următoare depinde doar de starea curentă

b) toate stările sunt la fel de probabile

c) starea următoare depinde de întreaga istorie

d) sistemul revine mereu în starea inițială

4. Care dintre următoarele NU poate fi o matrice de tranziție (rândurile = starea de plecare)?

a) [0,5 0,5; 0,5 0,5]

b) [1 0; 0 1]

c) [0 1; 1 0]

d) [0,7 0,4; 0,2 0,8]

5. Modelul vremii: soare → soare 0,9, ploaie → soare 0,5. Azi plouă. P(soare poimâine) =

a) 0,5

b) 0,7

c) 0,86

d) 0,9

6. Distribuția după n pași a unui lanț Markov se calculează cu:

a) $\pi^{(n)} = n \cdot \pi^{(0)}P$

b) $\pi^{(n)} = P^n \cdot \pi^{(0)}$ (vector-coloană, P pe coloane)

c) $\pi^{(n)} = \pi^{(0)} \cdot P^n$ (vector-linie)

d) $\pi^{(n)} = \pi^{(0)} + nP$

7. Distribuția staționară π a unui lanț ergodic reprezintă:

a) probabilitatea stării inițiale

b) fracțiunea de timp petrecută în fiecare stare pe termen lung

c) probabilitatea de a rămâne în stare la un pas

d) numărul de stări absorbante

8. Pietonul pe 4 colțuri (sens orar sau trigonometric cu ½) este un lanț:

a) absorbant

b) periodic, cu perioada 2

c) reductibil

d) aperiodic

9. O stare i este absorbantă dacă:

a) $\pi_i = 0$

b) $P_{ii} = 0$

c) e accesibilă din toate stările

d) $P_{ii} = 1$

10. Pentru un lanț absorbant, matricea fundamentală $N = (I - Q)^{-1}$ dă:

a) numărul mediu de vizite în stările tranzitorii

b) distribuția staționară

c) perioada lanțului

d) matricea de tranziție după 2 pași

11. În PageRank cu d = 0,85, cu ce probabilitate sare internautul la o pagină aleasă uniform, la fiecare pas?

a) 0,85

b) 0,15

c) 0,5

d) 1/n

12. De ce se adaugă teleportarea în PageRank?

a) ca să accelereze calculul matricei M

b) ca să crească scorul paginilor populare

c) ca lanțul să fie ireductibil și aperiodic, cu π unică

d) ca să elimine paginile fără linkuri

13. Un model de limbaj cu bigrame estimează P(b | a) prin:

a) nr(a urmat de b) / nr(a)

b) nr(b) / nr(a)

c) nr(a urmat de b) / nr. total de cuvinte

d) o rețea neurală

14. În algoritmul Metropolis, f(x) = 0,20 și f(y) = 0,05. Probabilitatea de a accepta propunerea y este:

a) 0

b) 0,05

c) 0,25

d) 1

15. Care afirmație despre eșantioanele obținute cu MCMC este corectă?

a) sunt independente

b) nu depind de punctul de start

c) se folosesc toate, inclusiv primele

d) sunt corelate; primele se aruncă (perioada de încălzire)

16. O plimbare 1D face 100 de pași, cu probabilitatea p = 0,6 de a urca. Care este poziția medie după 100 de pași?

(Răspuns numeric)

17. Joc corect (p = ½): pornim cu 30 de lei, pariem câte 1 leu și ne oprim la 0 sau la 100 de lei. Probabilitatea de a ajunge la 100 este:

(Răspuns numeric)

18. Un lanț cu două stări are P(A → B) = 0,2 și P(B → A) = 0,3. Ce fracțiune din timp petrece în A, pe termen lung?

(Răspuns numeric)

19. Un serviciu are MTBF = 99 de zile și MTTR = 1 zi. Care este disponibilitatea lui, în procente?

(Răspuns numeric)

20. Într-un corpus, cuvântul „date” apare de 50 de ori și este urmat de „reale” de 20 de ori. Cât este $\hat{P}$(„reale” | „date”) în modelul cu bigrame?

(Răspuns numeric)

Răspunsuri

1. b) – $\sigma = \sqrt{n} = \sqrt{400} = 20$.

2. c) – $\langle r^2 \rangle = 4Dt$ crește liniar în t: de 4 ori mai mult timp → 320.

3. a) – $P(X_{n+1} = j \mid X_0, \ldots, X_n) = P(X_{n+1} = j \mid X_n)$: trecutul nu aduce informație în plus.

4. d) – Primul rând al matricei D are suma 1,1; fiecare rând trebuie să aibă suma 1.

5. b) – 0,5·0,9 + 0,5·0,5 = 0,7.

6. c) – Cu convenția „rândul = de unde plecăm”, vectorul-linie se înmulțește la dreapta cu P^n.

7. b) – $\pi = \pi P$; π_i este fracțiunea de timp în starea i pe termen lung (teorema ergodică).

8. b) – Alternează colțurile pare și impare: revenirile sunt posibile doar la pași pari, deci $\pi^{(n)}$ oscilează.

9. d) – Odată intrat în starea i, procesul nu mai iese: $P_{ii} = 1$.

10. a) – N_{ij} = numărul mediu de vizite în j pornind din i; B = NR dă probabilitățile de absorbție.

11. b) – Cu d = 0,85 urmează un link, cu 1 − d = 0,15 se teleportează.

12. c) – Fără teleportare, capcanele și fundăturile fac lanțul reductibil; G are toate elementele > 0.

13. a) – Numărăm tranzițiile din a în b și împărțim la numărul de apariții ale lui a – ca la estimarea oricărei matrice de tranziție.

14. c) – $\min(1, f(y)/f(x)) = \min(1, 0{,}25) = 0{,}25$.

15. d) – Stările consecutive ale unui lanț sunt corelate, iar primele depind de start.

16. 20 – $E[X_n] = n(2p - 1) = 100 \cdot 0{,}2 = 20$.

17. 0,3 – $P = k/N = 30/100 = 0{,}3$.

18. 0,6 – $\pi_A = b/(a + b) = 0{,}3/0{,}5 = 0{,}6$.

19. 99 – $A = MTBF/(MTBF + MTTR) = 99/100 = 99\%$.

20. 0,4 – $20/50 = 0{,}4$.

Bibliografie

- Ș. Holban – Tehnici de modelare și simulare, cap. 5 (Modele Markov) și prezentarea „Markov Chains” (materiale de curs).
- Cernazean – Modelling and Simulation, Lecture 3: Dynamical systems (plimbarea aleatoare, difuzia) (materiale de curs).
- O. Häggström – Finite Markov Chains and Algorithmic Applications, Cambridge University Press, 2002.
- S. M. Ross – Introduction to Probability Models, Academic Press – capitolul despre lanțuri Markov.
- R. J. Gibbens – Computer Systems Modelling, University of Cambridge – lanțuri Markov și procese naștere–moarte.
- S. Brin, L. Page – The Anatomy of a Large-Scale Hypertextual Web Search Engine, Computer Networks and ISDN Systems, 1998.
- C. E. Shannon – A Mathematical Theory of Communication, Bell System Technical Journal, 1948.
- B. Hayes – First Links in the Markov Chain, American Scientist, 101(2), 2013.
- N. Metropolis, A. Rosenbluth, M. Rosenbluth, A. Teller, E. Teller – Equation of State Calculations by Fast Computing Machines, Journal of Chemical Physics, 1953.
- P. Diaconis – The Markov Chain Monte Carlo Revolution, Bulletin of the American Mathematical Society, 46(2), 2009.
- J. Ho, A. Jain, P. Abbeel – Denoising Diffusion Probabilistic Models, NeurIPS, 2020.
- Material realizat cu ajutorul Claude Fable 5.1 și Claude Opus 5.5.